

Modal Synthesis of a Glockenspiel: Comparison and Critiques

By Alex Ryström

February 2026

1 Introduction

The objective of this study was to analyse an audio recording of the Glockenspiel (specifically the note F#5) and reconstruct it using additive synthesis. By decomposing the recorded sound into its partials, we tried to show whether a sum of exponentially decaying sinusoids is enough to replicate the complex timbre of a percussion instrument.

2 Analysis

The majority of work was done using Python to analyse two recordings: a "Soft" strike (p) and a "Hard" strike (f).

2.1 Time Domain Visualisation

Before analysing the spectra, the raw waveforms were examined to understand the envelope of the signal. The hard strike showed a significantly sharper attack and higher initial peak amplitude compared to the soft strike.

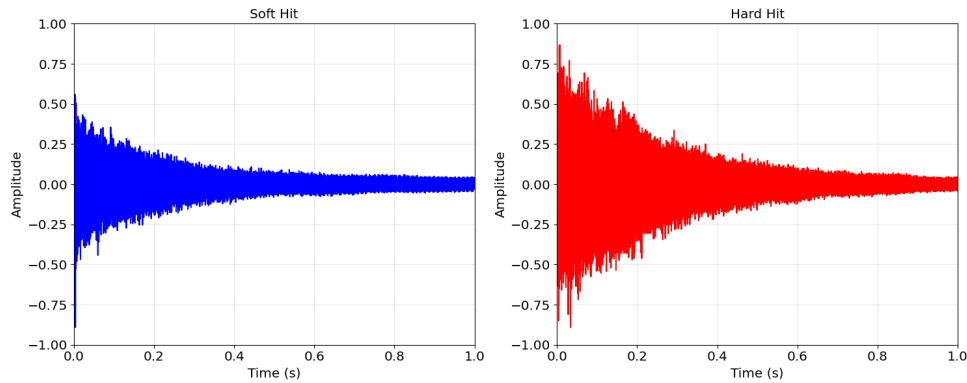


Figure 1: Time Domain Waveforms < 1 sec

2.2 Spectral Analysis

To identify the modal frequencies, a Fast Fourier Transform (FFT) was performed using a rectangular window of 100 ms with zero padding. Two snapshots were taken:

1. **Attack Phase** ($t = 0.0s$): To capture initial energy.
2. **Decay Phase** ($t = 0.7s$): To measure remaining energy after damping.

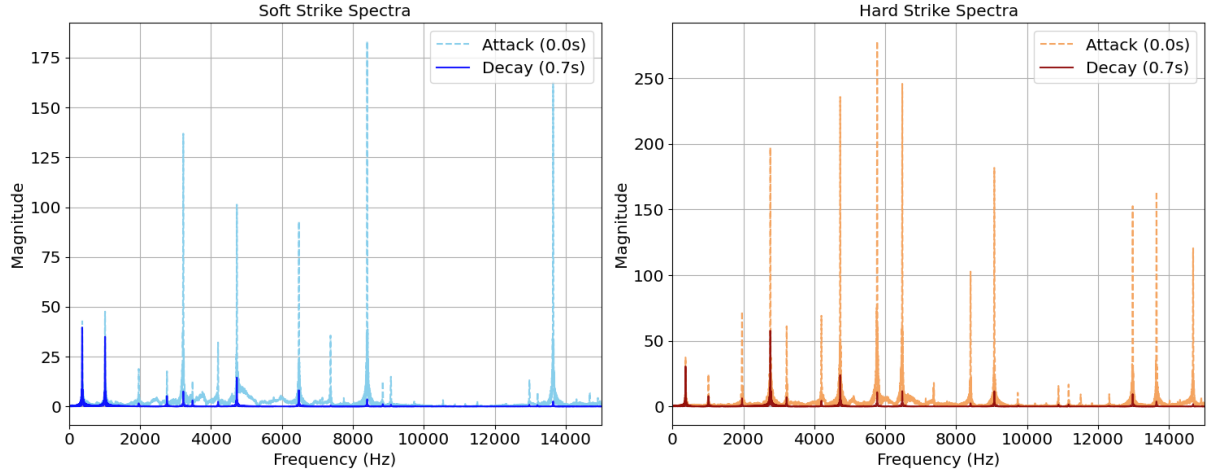


Figure 2: Linear Spectra Plot for Soft and Hard Strike, at 0s and 0.7s

The peaks were visualised in decibels to better identify relevant components. Significant partials were identified by picking peaks with threshold > -22 dB.

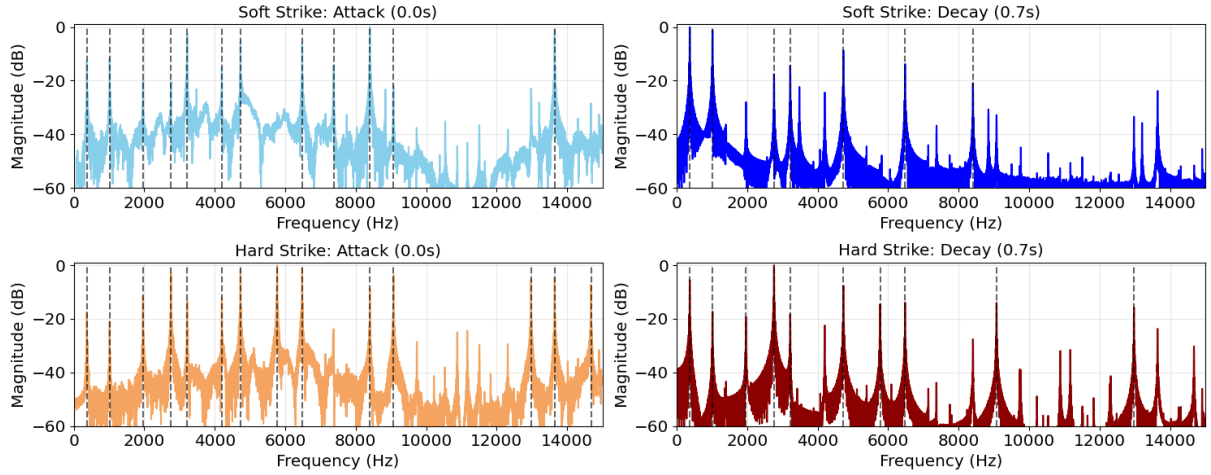


Figure 3: dB Spectra Plot for Soft and Hard Strike, at 0s and 0.7s, peaks labelled

2.3 Envelope Estimation

The amplitude envelope of each partial was assumed to follow a simple exponential decay law $A(t) = A_0 e^{-\gamma t}$. The decay rate γ for each partial was calculated by comparing the logarithmic magnitude (dB) difference between the attack and decay snapshots. Additionally, negative gamma values were handled to reduce specific modes from ringing out exponentially louder with time.

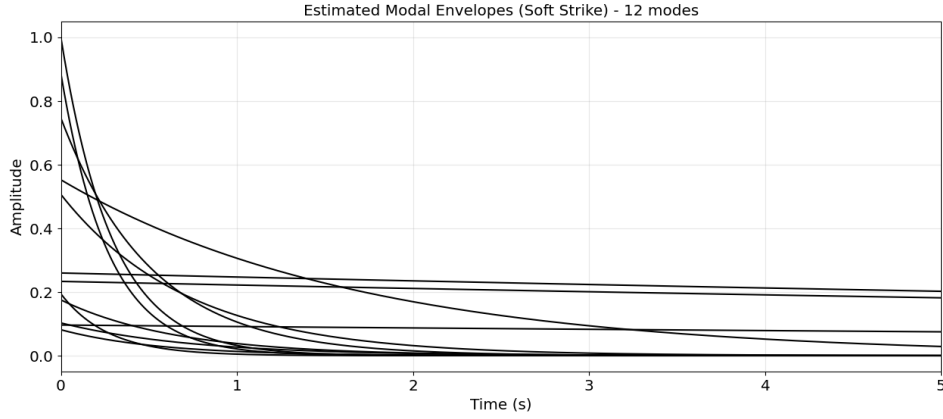


Figure 4: Envelope for Soft Strike

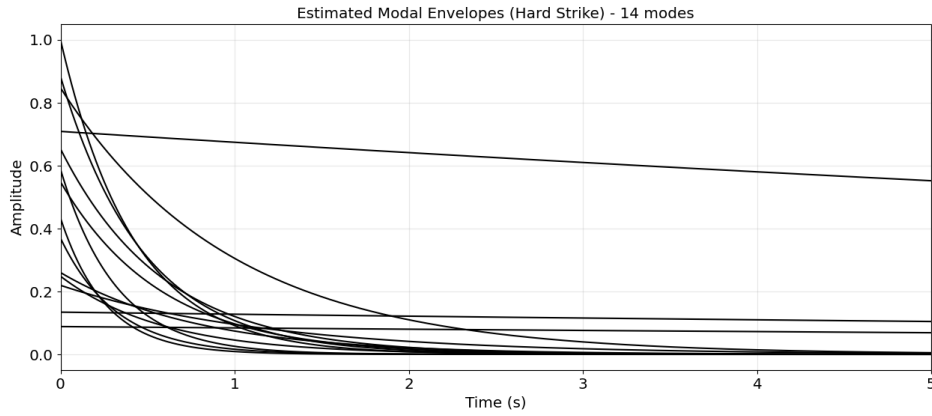


Figure 5: Envelope for Hard Strike

3 Synthesis and Playback Implementation

The final stage of the process involved generating the time domain signal from the analysed parameters and preparing it for audio output.

3.1 Additive Synthesis Logic

The synthesis was performed by iterating through the detected peaks and generating a decaying sinusoid for each mode. The code utilises the calculated amplitude (A), decay rate (γ), and phase (ϕ) to reconstruct the signal:

```
for ii in peaks_atk_hard:
    amp1 = MAG_ATK_HARD_DB[ii]
    amp2 = MAG_DEC_HARD_DB[ii]
    gamma = (np.log(10)/20) * (amp1 - amp2) / dt
    if gamma <= 0: gamma = 0.05 #handle to remove over sustained modes
    A = np.power(10, amp1/20) * np.power(10, gamma * t_attack/np.log10(np.exp(1)))
    t = np.arange(0, tonedur*gsp_hard.frame_rate)

    envelope = A*np.exp(-gamma*t/gsp_hard.frame_rate)
    ysum_hard += envelope*np.cos(2*np.pi*freqs_atk_hard[ii]*n/gsp_hard.frame_rate
                                + np.angle(HARDDATA_ATK[ii]))
```

3.2 Concatenation and Normalisation

To listen to differences clearer, the real audio was concatenated with the synthesised one (for both hard strike and soft strike). Before playback, the signal was normalised and converted to 16-bit integers:

```
ysum_hard /= np.max(np.abs(ysum_hard)) #normalise
ysum_hard = np.concatenate((harddata[0:len(ysum_hard)], ysum_hard))
ysum_scaled_hard = np.int16(ysum_hard / np.max(np.abs(ysum_hard)) * (np.power(2,16)/2))
write("gsp_hardstrike#5_synthesis_concatenated.wav", gsp_hard.frame_rate, ysum_scaled_hard)
play_obj = sa.play_buffer(ysum_scaled_hard, 1, 2, gsp_hard.frame_rate)
play_obj.wait_done()
```

A short melody was then made by adjusting the frequency of the synthesised note by a desired factor:

```
#format: (Semitone Shift, Duration)
#0 is original pitch. 2 is whole step up, etc.
melody_notes = [
    (2, 0.4),
    (0, 0.5),
    (2, 0.4),
    (-2, 0.6),
    (2, 0.3),
    (-2, 0.3),
    (0, 0.3),
    (2, 0.3), #here comes the sun..
]
full_melody_audio = np.array([])
for note in melody_score:
    semitone, dur = note
    #generate the note
    note_audio = synthesize_note(semitone, dur)
    #append to track
    full_melody_audio = np.concatenate((full_melody_audio, note_audio))
```

4 Comparison of Synthesis vs. Recording

4.1 Discrepancies in Audio Importation

An unexpected observation during the analysis was that the original audio, when imported into the Python environment, did not sound 100% accurate compared to listening to the raw `.wav` file directly through a standard media player. This maybe suggests potential issues with bit-depth conversion or sample rate handling during the array conversion process. Changes like this make it difficult to achieve a perfect synthesis since the model is trying to replicate a slightly different version of the instrument.

4.2 The Plain Quality

While the synthesis successfully captured the fundamental pitch and primary overtones, the result still felt quite plain. It was missing the "shimmer" and harmonic richness that was there in the original recording. By reducing the audio to 10+ static sine waves, the sound lost its depth and felt more like a cheap synthesizer than a physical instrument.

5 Critique and Technical Challenges

5.1 The Exponential Model

The assumption that the envelope is purely exponential ($e^{-\gamma t}$) is an over simplification. Real percussion instruments often show non-linear damping. Furthermore, a real glockenspiel has imperfections that cause interference. This simplification cannot reproduce amplitude fluctuations (tremolo) that is heard.

5.2 The Two-Point Estimation

The method of capturing just two spectral snapshots (at 0ms and 700ms) created a confusing technical obstacle in the estimation of the decay γ .

- **Exponential Increase:** For certain noisy partials or areas where spectral leakage occurred, the magnitude at 700ms was occasionally measured higher than at the attack.
- **Handling Errors:** Without handling it, the formula would produce a negative γ , causing the mode to increase in volume exponentially, essentially bombing the sound with an extremely loud tone over time.
- **Correction:** In the code, these occurrences had to be manually handled (I set it to a value of 0.05) to ensure the modes were audible but did not grow unreasonably. This shows that the two-snapshot method is sensitive to local fluctuations and possibly noise.

5.3 The Rectangular Window

The use of a rectangular window in the FFT analysis caused some spectral leakage. In the real instrument, close frequencies often interfere to create a shimmering/beating effect. When the signal is cut by a finite window, it causes energy to leak into other frequencies. The fault of a rectangular window was very evident when a Hanning window was used as a test, which provided a much clearer spectrum.

6 Suggested Improvements

6.1 Using More Measurement Points

The current "two-snapshot" method is very sensitive; if the sound happens to be slightly louder or quieter at exactly 700ms due to noise, the decay calculation fails and the volume explodes.

- **Proposal:** Instead of looking at only two points, we could measure the volume at 10 or 20 points throughout the sound and find the best fit line for the decay.
- **Benefit:** This would make the synthesis much more stable and prevent the need to manually "fix" decay values in the code.

6.2 Better Windowing

The rectangular window used in the analysis causes energy to leak between frequencies, which can hide the smaller, more delicate details of the sound.

- **Proposal:** Using a "Hanning" window (which fades the edges of the audio snapshot) would produce sharper peaks in the spectrum.
- **Benefit:** This would allow one, when selecting peaks using a threshold, to find frequencies which would otherwise be buried by spectral leakage.

6.3 Varying the Mallet Sound for Melodies

Currently, the melody sounds like the same recording being stretched.

- **Proposal:** On a real instrument, higher notes decay faster and have "tighter" impact sounds than low notes.
- **Benefit:** Adjusting the decay speed and the length of the attack noise based on the pitch would make the melody sound like a real set of Glockenspiel bars rather than a digital sample.

7 Conclusion

The additive synthesis model somewhat successfully reproduced the tonal center of the Glockenspiel, creating a recognisable pitch and melody. However, the results show that a purely mathematical approach, using only simple exponentially decaying sine waves, misses some key aspects which define the instrument. The plain quality of the synthesis and other challenges, such as spectral leakage and unstable decay estimates, prove that a realistic sound requires more than just the right frequencies.